

FastSD: An Efficient Cloud-Edge Collaborative Inference Framework with Speculative Decoding

Yunqi Gao, *Student Member, IEEE*, Bing Hu, *Senior Member, IEEE*,
Mahdi Boloursaz Mashhadi, *Senior Member, IEEE*, Pei Xiao, *Senior Member, IEEE*, and
Rahim Tafazolli, *Fellow, IEEE*

Abstract—Speculative Decoding

Index Terms—Collaborative Inference, Large Language Models, Speculative Decoding.

1 INTRODUCTION

COLLABORATIVE Inference

(1) *Efficient scheduling for multi-type, multi-length tasks from heterogeneous edge devices.* Edge devices continuously upload diverse tasks to the cloud-based target LLM, and these tasks differ in type (e.g., prefill or verify), size, and urgency. Moreover, edge devices are often heterogeneous in computing capability, communication bandwidth, and draft LLM configuration. An efficient scheduling strategy must adapt to task characteristics and device diversity.

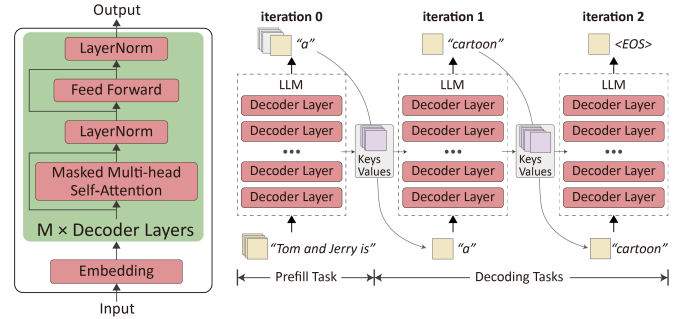
(2) *Batching and switching between prefill and verify under variable task loads.* Verify tasks are typically lightweight but frequent, while prefill tasks are few but heavy, and they are often delayed due to batch inefficiencies. Switching scheduling randomly between these task types can lead to GPU resource underutilisation or resource scarcity. Furthermore, batching decisions must adapt to real-time task load distribution including the number of tasks and sequence size changes.

(3) *Latency overhead from frequent KV cache I/O between disk and GPU.* To serve many edge requests concurrently, KV cache is typically stored on CPU or disk due to limited GPU memory. However, the repeated loading of past key-values for every verify task incurs non-trivial I/O latency, becoming a bottleneck especially in high-throughput scenarios. Existing collaborative inference based on speculative decoding usually ignores this overhead since they only consider a single edge device.

2 BACKGROUND AND CHALLENGES

2.1 Decoder-based Generative LLMs

Decoder-based Generative LLM Architecture. The decoder-based generative LLM adopts a stack of transformer decoder layers applied to embedded input tokens. As shown in Fig. 1a, the architecture begins with an



(a) LLM Architecture

(b) Autoregressive Inference

Fig. 1: The decoder-based LLM architecture and the generative inference procedure of an LLM.

embedding layer that maps input tokens into continuous representations, followed by M sequential decoder layers. Each decoder layer is composed of masked multi-head self-attention, a feed-forward network, and two LayerNorm modules. This architecture is widely adopted in modern LLMs such as GPT-3, LLaMA, and Deepseek-LLM due to its simplicity and efficiency in sequence generation tasks.

Autoregressive Inference. LLMs adopt an autoregressive inference strategy, where output tokens are generated sequentially, one token at a time. As illustrated in Fig. 1b, this process usually consists of two types of tasks: a prefill task and multiple decoding tasks. For the prefill task (iteration 0), the model processes the full input prompt (e.g., "Tom and Jerry is") to calculate hidden representations and initialize the KV cache for each decoder layer. For the subsequent decoding tasks, the model generates one token per step (e.g., "a", "cartoon", "<EOS>"), where each new token is appended to the previous context and used as the input for the next iteration. Crucially, the decoder layers reuse the stored KV cache to avoid recalculating the entire context, significantly accelerating inference. This step-by-step token generation continues until an end-of-sequence (<EOS>) token is produced. While this autoregressive mechanism enables coherent and context-aware generation, it introduces

- Yunqi Gao and Bing Hu are with Zhejiang University, Hangzhou 310027, China. E-mail: gaoyunqi1999, binghu@zju.edu.cn.
- Mahdi Boloursaz Mashhadi, Pei Xiao and Rahim Tafazolli are with 5GIC & 6GIC, Institute for Communication Systems (ICS), University of Surrey, United Kingdom. E-mail: m.boloursazmashhadi, p.xiao, r.tafazolli@surrey.ac.uk.

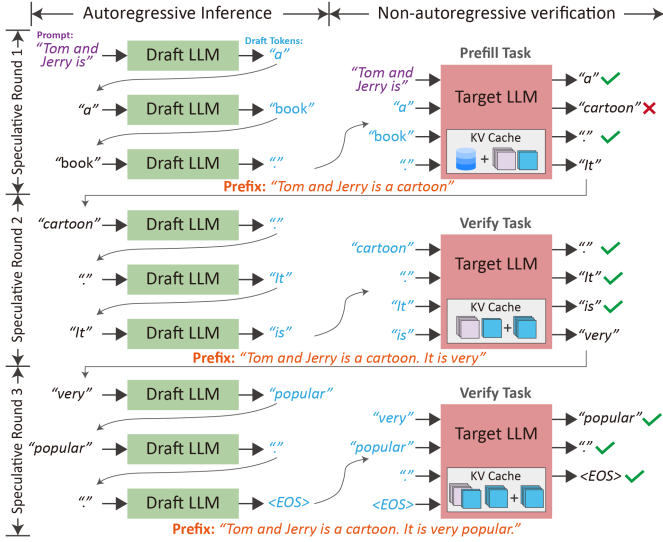


Fig. 2: An example to show the workflow of speculative decoding.

inherent computational dependencies that limit parallelism during inference.

2.2 Speculative Decoding

Speculative decoding is an efficient decoding strategy designed to accelerate LLM inference by leveraging two models of different sizes. As shown in Fig. 1b, traditional autoregressive inference requires the target LLM to generate tokens sequentially, with each new token dependent on previously generated ones. The process is inherently time-consuming due to its strict sequential nature. In contrast, speculative decoding introduces a small draft LLM to propose candidate tokens (*draft tokens*), while a large target LLM serves as a verifier. The draft LLM generates multiple tokens through autoregressive inference, and the target LLM subsequently performs *non-autoregressive verification* (NAV) on the entire sequence generated by the draft LLM in one forward pass.

Fig. 2 illustrates an example of a speculative decoding workflow. First, the draft LLM receives a prompt of “Tom and Jerry is” and performs three autoregressive inferences (which we call the *draft length*) to generate “Tom and Jerry is a book.” Then, the target LLM takes “Tom and Jerry is a book.” as input and simultaneously obtains all outputs (the next token) for each token in the sequence, i.e., “a” for “Tom and Jerry is”, “cartoon” for “a”, “.” for “book” and “It” for “.”. The process where the draft model generates a batch of draft tokens and the target model verifies them is defined as a *Speculative Round*. The sequence that has already been verified and accepted by the target LLM is referred to as the *prefix*, which serves as the input for the draft LLM in the next speculative round. During this round, the target LLM uses its own predictions as ground truth to identify and correct any deviations in the draft LLM’s outputs. For example, the “book” generated by the draft LLM is incorrect and should be corrected to “cartoon” based on the target LLM’s outputs. Finally, accepted tokens including all correct and corrected

TABLE 1: The probability of correct tokens draft LLM can generate with the target LLM verification.

Dataset (Task)	Draft LLM	Target LLM	Correctness
MT-Bench (QA)	TinyLlama-1.1B-Chat-v0.3	Llama-2-7B	65.1%
	TinyLlama-1.1B-Chat-v0.3-platypus		74.7%
GSM8K (Math)	TinyLlama-1.1B-Chat-v1.0	Llama-2-7B	71.5%
	TinyLlama-1.1B-1T-OpenOrca		75.4%
Humaneval (Code Generation)	Deepseek-coder-1.3B	Deepseek-coder-6.7B	86.3%
	TinyLlama-1.1B-python-v0.1		74.3%

tokens are sent to the draft LLM for the next speculative round.

After the target LLM completes NAV, the KV cache of accepted tokens is generated and saved. This KV cache is directly reused in subsequent verifications. Especially, the task of generating the KV cache of prompt is called the *prefill task* (i.e., the NAV task performed by the target LLM in speculative round 1), while the remaining are called the *verify tasks*. Therefore, prefill tasks are more time-consuming, and verify tasks are more numerous.

Speculative decoding accelerates inference for two core reasons: **(1) Draft LLMs demonstrate strong performance on common tasks.** We define correctness as the probability that a draft LLM generates the same tokens as the target LLM. Table 1 shows the correctness of six draft LLMs across three inference tasks with two target LLM verification, where incorrect tokens generated by the draft LLM are promptly corrected and reused as input for subsequent autoregressive inference. Results indicate that all draft LLMs correctly generated more than 65% tokens, with draft Deepseek-coder LLM exhibiting an impressive 86.3% accuracy on the code generation task. **(2) The NAV of the target LLM examines draft sequences by only one forward pass, resulting in high computational efficiency and device utilization.** Unlike traditional autoregressive inference that processes tokens sequentially and requires repeated access to model parameters, NAV enables the target LLM to verify the entire draft sequence through a single forward pass. This parallelism significantly reduces computational redundancy, thereby enhancing inference speed and computational resource utilization.

2.3 Cloud-edge Collaborative Inference

Speculative decoding is typically applied to collaborative inference between cloud and edge. As shown in Fig. 3, the draft LLM is deployed on edge device (e.g., smartphones, tablets, etc) while the target LLM resides in the cloud (e.g., data centers). A speculative round primarily consists of two steps: (1) the edge draft LLM performs autoregressive token generation and transmits the token sequence to the cloud; (2) the cloud target LLM executes NAV and returns all accepted tokens. This design offers three key advantages: (1) reducing cloud workload by offloading token generation

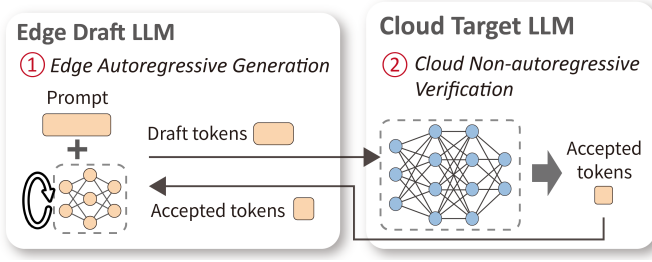


Fig. 3: Cloud-edge collaborative inference workflow with single draft LLM.

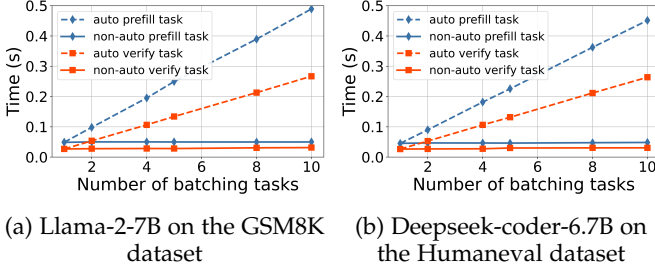


Fig. 4: The inference time with the number of batching tasks increasing. ‘auto’: autoregressive verification; ‘non-auto’: non-autoregressive verification.

of draft LLM to edge devices; (2) enabling partial on-device inference through edge deployment, thereby minimizing cloud-edge communication latency and enhancing privacy; (3) allowing users to perform simple tasks using the edge draft LLM even when no network or poor connectivity is available.

2.4 Motivation and Challenges

Existing speculative decoding frameworks for cloud-edge collaboration have limited applicability, as they typically consider only a single edge draft LLM collaborating with a cloud target LLM. This setup is often impractical, since in real-world scenarios, the cloud target LLM usually receives inference requests from thousands of edge draft LLMs over time. Enabling collaboration among multiple edge draft LLMs through a single cloud target LLM allows parallel draft generation across distributed edge devices, thereby enhancing the scalability and efficiency of collaborative inference. However, implementing a cloud-edge collaborative inference framework with multiple draft LLMs faces the following three challenges:

(1) **Efficient scheduling for multi-type, multi-length tasks from heterogeneous edge devices.** Edge devices continuously upload diverse tasks to the cloud-based target LLM, and these tasks differ in type (e.g., prefill or verify), size, and urgency. Moreover, edge devices are often heterogeneous in computing capability, communication bandwidth, and draft LLM configuration. An efficient scheduling strategy must adapt to task characteristics and device diversity.

(2) **Batching and switching between prefill and verify under variable task loads.** Batching is a widely adopted strategy in LLM inference frameworks because it allows

multiple decoding tasks to be processed simultaneously in a single forward pass, which can improve GPU utilization and reduce the latency of each task. We evaluate the batching time of two target LLMs across different inference modes and tasks. We randomly select a task from each dataset and replicate it into different numbers to send concurrently to the target LLM. From Fig. 4, NAV significantly reduced verification time for Llama-2-7B and Deepseek-coder-6.7B models by 8.6–9.8× across 10 tasks. However, when the sequence lengths of batching tasks vary significantly, padding is required to align them, which introduces redundant computations on padded tokens and reduces overall efficiency. Therefore, batching decisions must adapt to real-time task load distribution including the number of tasks and sequence size changes. Moreover, verify tasks are typically lightweight but frequent, while prefill tasks are few but heavy. A large number of verify tasks may accumulate due to the long processing time of prefill tasks, increasing latency. In contrast, over-prioritizing verify tasks can lead to starvation of prefill tasks, reducing overall throughput. Therefore, an adaptive task scheduling switching strategy is crucial.

(3) **Latency overhead from frequent KV cache I/O between disk and GPU.** To serve many edge requests concurrently, KV cache is typically stored on CPU or disk due to limited GPU memory. However, the repeated loading of past key-values for every verify task incurs non-trivial I/O latency, becoming a bottleneck especially in high-throughput scenarios. Existing collaborative inference based on speculative decoding usually ignores this overhead since they only consider a single edge device.

3 FASTSD DESIGN

3.1 Overview

3.2 Priority-based Multi-queue Task Scheduling

The cloud target LLM receives a large number of heterogeneous NAV tasks from edge draft LLMs, and the scheduling order of these NAV tasks in the cloud significantly affects the total completion time of all inference tasks. We define that the heterogeneity of a NAV task is determined by five factors: (1) the computing power of the edge device, (2) the communication conditions of the edge device, (3) the correctness of the draft LLM, (4) the token length of the sequence to be verified, and (5) the type of NAV task. We then designed a priority-based multi-queue task scheduling mechanism to order all NAV tasks. Firstly, since prefill tasks typically take much longer than verify tasks, we divide all tasks into two groups based on their type: prefill task group and verify task group. Secondly, we construct three queues for each task type based on the token length of the sequence to be verified: tasks with token length $\leq R_1$ are placed in the “short task” queue, tasks with token length between R_1 and R_2 are placed in the “medium task” queue, and tasks with token length $> R_2$ are placed in the “long task” queue. In practical cloud-edge collaborative system, since the edge draft LLMs may upload NAV tasks to the cloud at any time and the prefix length of each task continuously increases, we cannot precisely predict the token length distribution of NAV tasks. Therefore, we set initial thresholds for R_1 and

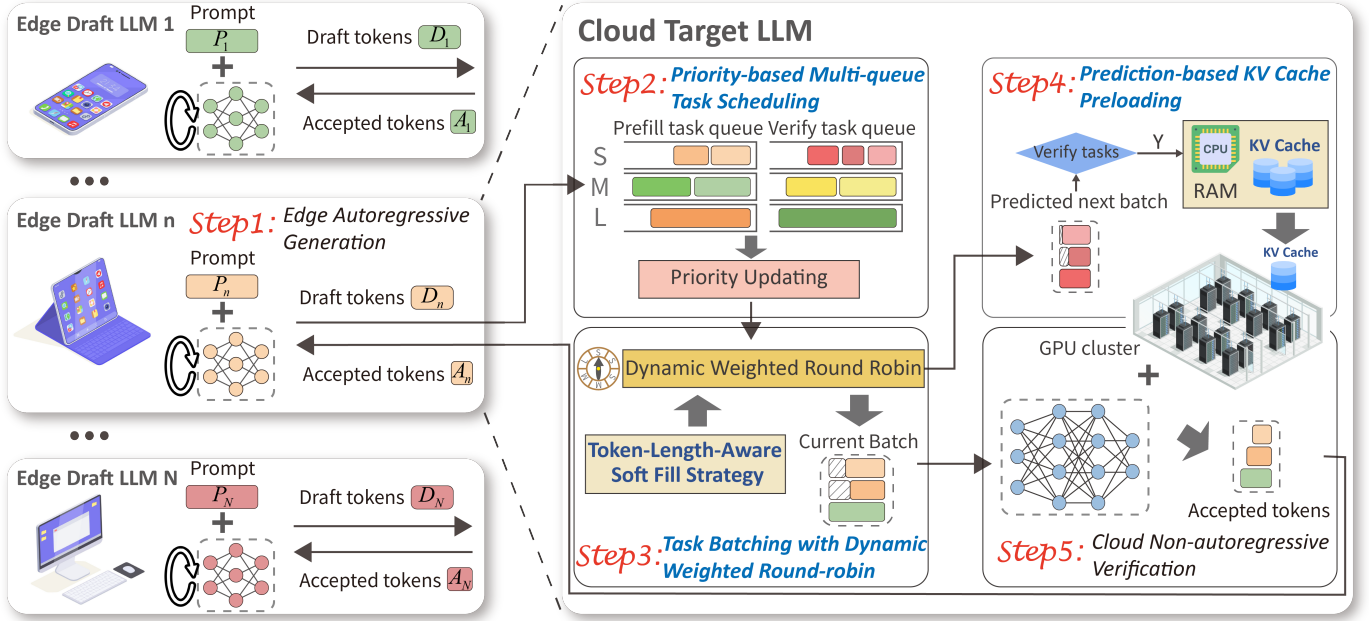


Fig. 5: The workflow of FastSD.

R_2 (e.g., $R_1 = 128$ and $R_2 = 512$), and the system dynamically adjusts these thresholds based on runtime statistics. Specifically, the cloud monitors the token length distribution of recent tasks within a sliding window (e.g., 100 cloud NAVs) and updates the boundaries for short, medium, and long tasks based on quantile estimates (e.g., the 70th and 90th percentiles).

3.3 Task Batching with Dynamic Weighted Round Robin

Sliding Window Based Scheduling Utilization-aware Triggering.

3.4 Prediction-based KV Cache Preloading